

# Contents

|                                          |          |
|------------------------------------------|----------|
| <b>cc-manage Setup and Usage Guide</b>   | <b>2</b> |
| Table of Contents                        | 2        |
| Prerequisites                            | 2        |
| Required                                 | 2        |
| Optional (for development/testing)       | 3        |
| Verify Prerequisites                     | 3        |
| Installation                             | 3        |
| Option A: One-Line Install (Recommended) | 3        |
| Option B: Manual Install from Git        | 3        |
| Verify Installation                      | 4        |
| First-Time Setup                         | 4        |
| Step 1: Run Doctor                       | 4        |
| Step 2: Add Your First Profile           | 4        |
| Step 3: Switch to Your Profile           | 5        |
| Step 4: Launch Claude Code               | 5        |
| Command Reference                        | 5        |
| Core Commands                            | 5        |
| Management Commands                      | 5        |
| Key Management Commands                  | 6        |
| Model Commands                           | 6        |
| Help Commands                            | 6        |
| Profile Management                       | 6        |
| Profile Files                            | 6        |
| Adding Profiles                          | 7        |
| Editing Profiles                         | 7        |
| Deleting Profiles                        | 7        |
| API Key Management                       | 7        |
| Key Storage                              | 7        |
| Key Naming Convention (V2)               | 8        |
| Managing Keys                            | 8        |
| Key Listing Output                       | 8        |
| Model Management                         | 8        |
| Default Models                           | 8        |
| Dynamic Model Refresh                    | 9        |
| Custom Models                            | 9        |
| Model Aliases                            | 9        |
| Proxy Architecture                       | 9        |
| When Proxies Are Used                    | 9        |
| How Proxies Work                         | 10       |
| Proxy Ports                              | 10       |
| Health Checks                            | 10       |
| Web Dashboard                            | 10       |
| Starting the Dashboard                   | 10       |
| Dashboard Sections                       | 11       |
| Security                                 | 11       |
| API Endpoints                            | 11       |

|                                    |    |
|------------------------------------|----|
| Troubleshooting . . . . .          | 12 |
| Common Issues . . . . .            | 12 |
| Running Tests . . . . .            | 13 |
| Uninstall . . . . .                | 13 |
| Backup First . . . . .             | 13 |
| Remove on Windows . . . . .        | 13 |
| Remove on macOS/Linux . . . . .    | 13 |
| Clean Up Claude Settings . . . . . | 13 |

## cc-manage Setup and Usage Guide

A complete, step-by-step guide to installing, configuring, and using cc-manage for provider-aware Claude Code profile management.

### Table of Contents

1. [Prerequisites](#)
2. [Installation](#)
3. [First-Time Setup](#)
4. [Command Reference](#)
5. [Profile Management](#)
6. [API Key Management](#)
7. [Model Management](#)
8. [Proxy Architecture](#)
9. [Troubleshooting](#)
10. [Uninstall](#)

### Prerequisites

Before installing cc-manage, ensure you have the following:

#### Required

| Tool               | Version                                 | Purpose                      | Install                                                    |
|--------------------|-----------------------------------------|------------------------------|------------------------------------------------------------|
| <b>Node.js</b>     | 16+                                     | Runs compatibility proxies   | <a href="https://nodejs.org">nodejs.org</a>                |
| <b>PowerShell</b>  | 5.1+ (Windows) or pwsh 7+ (macOS/Linux) | CLI engine                   | Pre-installed on Windows; brew install powershell on macOS |
| <b>Claude Code</b> | Latest                                  | The CLI tool cc-manage wraps | <code>npm install -g @anthropic-ai/claude-code</code>      |

## Optional (for development/testing)

| Tool          | Version | Purpose                        |
|---------------|---------|--------------------------------|
| <b>Python</b> | 3.8+    | Running proxy conversion tests |
| <b>Git</b>    | Any     | Cloning the repository         |

## Verify Prerequisites

```
# Check Node.js
node --version      # Should show v16+

# Check PowerShell (Windows)
$PSVersionTable.PSVersion  # Should show 5.1+

# Check PowerShell Core (macOS/Linux)
pwsh --version      # Should show 7+

# Check Claude Code
claude --version    # Should show installed version
```

## Installation

### Option A: One-Line Install (Recommended)

#### Windows PowerShell:

```
irm https://raw.githubusercontent.com/Shashank-1K/cc-manager/main/install.ps1 | iex
```

This will: 1. Download the latest cc-manage from GitHub 2. Extract to ~/.claude-profiles/ 3. Add ~/.claude-profiles to your user PATH 4. Run cc-manage doctor to verify installation

#### macOS/Linux:

```
curl -fsSL https://raw.githubusercontent.com/Shashank-1K/cc-manager/main/install.sh | sh
```

This will: 1. Download the latest cc-manage from GitHub 2. Extract to ~/.claude-profiles/ 3. Make shell launchers executable 4. Add PATH entry to your shell RC file (~/.zshrc, ~/.bashrc, or ~/.profile) 5. Run cc-manage doctor to verify installation

### Option B: Manual Install from Git

```
git clone https://github.com/Shashank-1K/cc-manager.git
cd cc-manage
```

#### Windows:

```
Copy-Item -Path "src\cc-manage\*" -Destination "$HOME\.claude-profiles" -Recurse -Force
New-Item -ItemType Directory -Force -Path "$HOME\.claude-profiles\profiles"
# Add to PATH (User Level)
$currentPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$currentPath;$HOME\.claude-profiles", "User")
```

### macOS/Linux:

```
mkdir -p ~/.claude-profiles/profiles
cp -R src/cc-manage/* ~/.claude-profiles/
chmod +x ~/.claude-profiles/ccs ~/.claude-profiles/cc-switch ~/.claude-profiles/cc-status
echo 'export PATH="$HOME/.claude-profiles:$PATH"' >> ~/.zshrc # or ~/.bashrc
source ~/.zshrc # or ~/.bashrc
```

## Verify Installation

```
cc-manage doctor
```

Expected output:

```
Claude Profiles Doctor
Node: /usr/local/bin/node
Claude Code: /usr/local/bin/claude
Env file: missing          # Normal if no keys configured yet
Doctor passed.
```

---

## First-Time Setup

After installation, follow these steps in order:

### Step 1: Run Doctor

```
cc-manage doctor
```

This checks: - Node.js is installed (needed for proxies) - Claude Code is installed - .env file exists - All profiles have valid API key references - All proxy scripts exist

### Step 2: Add Your First Profile

```
cc-manage add
```

This opens an interactive wizard:

1. **Select Provider** — Choose from the list (e.g., "Anthropic", "Gemini", "OpenRouter")
2. **Filename** — Enter a short name (e.g., "my-anthropic", "gemini-flash")
3. **Profile Display Name** — Human-readable name (defaults to filename)
4. **API Key** — Enter or select an existing key

5. **Base URL** — Confirm or change the provider endpoint
6. **Models** — Select from defaults or enter custom model names
7. **Default Model** — Choose which model to use by default

### Step 3: Switch to Your Profile

```
cc-switch
```

This shows a numbered list of profiles. Select one by number or name:

```
cc-switch my-anthropic      # By name
cc-switch 1                 # By number
cc-switch 1 gemini-2.5-flash # Profile + model
```

### Step 4: Launch Claude Code

```
ccs
```

This reads your active profile, starts any needed proxy, sets environment variables, and launches Claude Code.

## Command Reference

### Core Commands

| Command                     | Description                            | Example             |
|-----------------------------|----------------------------------------|---------------------|
| cc-switch                   | Interactive profile/model selector     | cc-switch           |
| cc-switch <profile> [model] | Non-interactive switch                 | cc-switch 3 1       |
| ccs [args]                  | Launch Claude Code with active profile | ccs --print "Hello" |
| cc-status                   | Show active profile details            | cc-status           |
| cc-dashboard                | Launch web management dashboard        | cc-dashboard        |
| claude                      | Alias for ccs                          | claude              |

### Management Commands

| Command                          | Description                 | Example                                 |
|----------------------------------|-----------------------------|-----------------------------------------|
| cc-manage                        | Interactive management menu | cc-manage                               |
| cc-manage add                    | Add a new profile           | cc-manage add                           |
| cc-manage edit [profile]         | Edit an existing profile    | cc-manage edit 2                        |
| cc-manage doctor                 | Check installation health   | cc-manage doctor                        |
| cc-manage test <profile> [model] | Test a profile              | cc-manage test my-profile --level tools |

| Command                         | Description                        | Example                         |
|---------------------------------|------------------------------------|---------------------------------|
| <code>cc-manage migrate</code>  | Migrate v1 profiles to v2 key IDs  | <code>cc-manage migrate</code>  |
| <code>cc-manage settings</code> | Fix Claude settings.json conflicts | <code>cc-manage settings</code> |
| <code>repair</code>             |                                    | <code>repair</code>             |

## Key Management Commands

| Command                                                                   | Description                            | Example                                                                 |
|---------------------------------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------|
| <code>cc-manage key list</code>                                           | Show all API keys with redacted values | <code>cc-manage key list</code>                                         |
| <code>cc-manage key set</code><br><code>&lt;KEY_ID&gt;</code>             | Add/update a key value                 | <code>cc-manage key set</code><br><code>CCKEY_GEMINI_MY_1234ABCD</code> |
| <code>cc-manage key rename</code><br><code>&lt;OLD&gt; &lt;NEW&gt;</code> | Rename a key                           | <code>cc-manage key rename</code><br><code>OLD_KEY NEW_KEY</code>       |
| <code>cc-manage key remove</code><br><code>&lt;KEY_ID&gt;</code>          | Remove a key from .env                 | <code>cc-manage key remove</code><br><code>OLD_KEY</code>               |

## Model Commands

| Command                                                                  | Description                     | Example                                                      |
|--------------------------------------------------------------------------|---------------------------------|--------------------------------------------------------------|
| <code>cc-manage models</code><br><code>&lt;provider&gt;</code>           | List available models           | <code>cc-manage models groq</code>                           |
| <code>cc-manage models</code><br><code>&lt;provider&gt; --refresh</code> | Fetch live models from provider | <code>cc-manage models groq</code><br><code>--refresh</code> |

## Help Commands

| Command                                | Description                           |
|----------------------------------------|---------------------------------------|
| <code>cc-manage -help</code>           | Show general help (interactive pages) |
| <code>cc-manage -help general</code>   | General help page                     |
| <code>cc-manage -help commands</code>  | Commands help page                    |
| <code>cc-manage -help uninstall</code> | Uninstall instructions                |

## Profile Management

### Profile Files

Profiles are stored as PowerShell scripts in `~/.claude-profiles/profiles/`:

```
~/.claude-profiles/profiles/my-anthropic.ps1
~/.claude-profiles/profiles/gemini-flash.ps1
```

Each profile contains:

```
$script:PROFILE_VERSION = 2
$script:PROFILE_NAME = "My Anthropic Profile"
$script:PROVIDER = "anthropic"
$script:MODE = "anthropic-direct"
$script:BASE_URL = "https://api.anthropic.com"
$script:AUTH_MODE = "api_key"
$script:API_KEY_ID = "CCKEY_ANTHROPIC_MY_1234ABCD"
$script:API_KEY_NAME = "CCKEY_ANTHROPIC_MY_1234ABCD"
$script:DEFAULT_MODEL = "claude-sonnet-4-20250514"
$script:MODELS = @("claude-sonnet-4-20250514", "claude-opus-4-20250514")
```

## Adding Profiles

### Interactive:

```
cc-manage add
```

### Provider-specific examples:

```
cc-manage add # Select Gemini → enter API key → choose gemini-2.5-flash
cc-manage add # Select OpenRouter → enter API key → choose anthropic/claude-sonnet-4
cc-manage add # Select Groq → enter API key → select dynamic models
```

## Editing Profiles

```
cc-manage edit          # Interactive selection
cc-manage edit 3        # Edit profile #3
cc-manage edit my-profile # Edit by name
```

Editable fields: - Profile display name - Provider and mode - Base URL and upstream URL - API key assignment - Model list (add/remove/set default)

## Deleting Profiles

```
cc-manage # Select "Delete Profile" from menu
```

Warning: Deleting a profile does NOT delete its API key from .env. Use `cc-manage key remove` to clean up unused keys.

## API Key Management

### Key Storage

API keys are stored in `~/.claude-profiles/.env` as environment variables:

```
CCKEY_GEMINI_MY_1234ABCD=AIza...
CCKEY_OPENROUTER_WORK_5678EFG=sk-or-...
ANTHROPIC_API_KEY=sk-ant-...
```

## Key Naming Convention (V2)

Keys use generated IDs: CCKEY\_<PROVIDER>\_<PROFILE>\_<RANDOM>

This means: - Profile files never contain raw API keys - Multiple profiles can share one key - Key values are stored only in .env

## Managing Keys

```
# List all keys
cc-manage key list

# Add a new key
cc-manage key set CCKEY_GEMINI_NEW_1234ABCD

# Rename a key
cc-manage key rename OLD_KEY_NAME NEW_KEY_NAME

# Remove a key
cc-manage key remove OLD_KEY_NAME
```

## Key Listing Output

```
--- Configured API Keys ---
- CCKEY_GEMINI_MY_1234ABCD (Provider: Gemini, Suffix: MY_1234ABCD)
  Value:  AIza...abcd
  Usage:  Used by: my-gemini-profile

- OPENROUTER_API_KEY (Provider: OpenRouter, Suffix: (default))
  Value:  sk-or...wxyz
  Usage:  Used by: openrouter-free
```

---

## Model Management

### Default Models

Each provider comes with default models. For example: - **Anthropic:** claude-sonnet-4-20250514, claude-opus-4-20250514 - **Gemini:** gemini-2.5-flash, gemini-3.5-flash - **Groq:** openai/gpt-oss-120b, qwen/qwen3-32b



## Dynamic Model Refresh

Some providers support fetching live model lists:

```
cc-manage models groq --refresh
cc-manage models mistral --refresh
cc-manage models mistral-vibe --refresh
cc-manage models nvidia-nim --refresh
```

## Custom Models

When adding or editing a profile, you can enter custom models:

```
cc-manage add
# ... select provider ...
# When prompted for models:
Models (comma separated, blank for suggested): my-custom-model, another-model
Default Model [my-custom-model]:
```

## Model Aliases

When launching Claude Code, ccs sets model aliases: - ANTHROPIC\_MODEL = selected model - ANTHROPIC\_DEFAULT\_SONNET\_MODEL = selected model (or first fallback) - ANTHROPIC\_DEFAULT\_OPUS\_MODEL = first non-selected model - ANTHROPIC\_DEFAULT\_HAIKU\_MODEL = second non-selected model

This means /model commands inside Claude Code will map to your chosen models.

---

## Proxy Architecture

### When Proxies Are Used

Not all providers need a proxy:

| Mode               | Providers                                  | Proxy Needed                                                  |
|--------------------|--------------------------------------------|---------------------------------------------------------------|
| anthropic-direct   | Anthropic, OpenRouter, DeepSeek, Fireworks | No                                                            |
| gemini-proxy       | Gemini                                     | Yes (anthropic-gemini-proxy.js)                               |
| openai-chat-proxy  | Groq, Together, xAI, Ollama Cloud, Custom  | Yes (openai-chat-proxy.js)                                    |
| mistral-proxy      | Mistral                                    | Yes (mistral-anthropic-proxy.js -> openai-chat-proxy.js)      |
| mistral-vibe-proxy | Mistral Vibe                               | Yes (mistral-vibe-anthropic-proxy.js -> openai-chat-proxy.js) |
| codestral-proxy    | Codestral                                  | Yes (codestral-anthropic-proxy.js -> openai-chat-proxy.js)    |
| nvidia-proxy       | NVIDIA NIM                                 | Yes (nvidia-anthropic-proxy.js -> openai-chat-proxy.js)       |

| Mode                    | Providers         | Proxy Needed                                         |
|-------------------------|-------------------|------------------------------------------------------|
| huggingface-proxy       | Hugging Face      | Yes (hug-anthropic-proxy.js -> openai-chat-proxy.js) |
| opencode-nemotron-proxy | OpenCode Nemotron | Yes (opencode-nemotron-proxy.js)                     |

## How Proxies Work

1. ccs starts the proxy on `127.0.0.1:<port>` (default ports 18000-18100)
2. Claude Code sends Anthropic Messages API requests to `http://127.0.0.1:<port>/v1/messages`
3. The proxy translates to the provider's native format (OpenAI Chat Completions, Gemini generateContent, etc.)
4. The proxy sends the translated request to the upstream provider
5. The proxy translates the response back to Anthropic Messages format
6. Claude Code receives a compatible response

## Proxy Ports

| Provider                   | Default Port  |
|----------------------------|---------------|
| Gemini                     | 18000         |
| NVIDIA NIM                 | 18003         |
| Hugging Face               | 18004         |
| Mistral                    | 18005         |
| Codestral                  | 18006         |
| Mistral Vibe               | 18007         |
| OpenCode Nemotron          | 18100         |
| OpenAI-compatible (custom) | User-selected |

## Health Checks

All proxies expose a health endpoint:

```
curl http://127.0.0.1:18000/health
# {"ok":true,"model":"gemini-2.5-flash"}
```

## Web Dashboard

cc-manager includes a local web dashboard for visual management.

## Starting the Dashboard

```
cc-dashboard # Default port 18200
cc-dashboard --port 19000 # Custom port
DASHBOARD_PORT=19000 cc-dashboard # Via environment variable
```

Open `http://127.0.0.1:18200` in your browser.

## Dashboard Sections

| Section             | What it does                                                                                                            |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Dashboard</b>    | Active profile, stat cards, proxy status grid with live indicators, system info. Auto-refreshes every 10 seconds.       |
| <b>Profiles</b>     | Table of all profiles with create/edit/delete. Modal forms for provider selection, key assignment, model configuration. |
| <b>API Keys</b>     | List all keys with redacted values. Shows which profiles use each key. Add/edit/delete with provider association.       |
| <b>Providers</b>    | Card grid of all 16 providers showing mode, port, auth type, and default models.                                        |
| <b>Proxy Health</b> | Live port status check for all proxy services (Gemini, Mistral, NVIDIA, etc.).                                          |
| <b>Models</b>       | Select a provider to view its models. Click to copy model names.                                                        |
| <b>Doctor</b>       | System health checks: Node.js, Claude Code, .env, per-profile key resolution, proxy script existence.                   |
| <b>Settings</b>     | Theme selector (light/dark/system), settings.json repair button.                                                        |

## Security

- Binds to `127.0.0.1` only — not accessible from other devices
- API key values are always returned redacted (first 4 + ... + last 4 chars)
- No npm dependencies — pure Node.js built-in modules
- Reads and writes the same data files as the CLI (fully interchangeable)

## API Endpoints

The dashboard exposes a REST API under `/api/`:

| Method              | Endpoint                           | Description                                                 |
|---------------------|------------------------------------|-------------------------------------------------------------|
| GET                 | <code>/api/overview</code>         | Dashboard summary with active profile, counts, proxy status |
| GET                 | <code>/api/providers</code>        | Full provider registry                                      |
| GET/POST/PUT/DELETE | <code>/api/profiles[/:name]</code> | Profile CRUD                                                |
| GET/POST/PUT/DELETE | <code>/api/keys[/:name]</code>     | API key CRUD                                                |
| GET                 | <code>/api/health</code>           | Comprehensive health checks                                 |
| GET                 | <code>/api/doctor</code>           | Doctor checks                                               |
| GET                 | <code>/api/models/:provider</code> | Provider model list                                         |
| GET/PUT             | <code>/api/theme</code>            | Theme management                                            |
| POST                | <code>/api/settings/repair</code>  | Fix settings.json conflicts                                 |

---

## Troubleshooting

### Common Issues

```
cc-switch    # Select a profile first
ccs          # Then Launch
```

#### “No active profile found”

“Missing API key id” The profile references a key that doesn’t exist in .env:

```
cc-manage key list          # See what keys exist
cc-manage key set <KEY_ID>  # Add the missing key
# Or edit the profile to use an existing key:
cc-manage edit <profile>
```

“Claude Code executable not found” Set CLAUDE\_CODE\_BIN to the full path:

```
$env:CLAUDE_CODE_BIN = "C:\Users\you\.local\bin\claude.exe"
# Or add to system environment permanently
```

Proxy won’t start / port already in use Another process is using the proxy port:

```
# Find what's using the port (Windows)
Get-NetTCPConnection -LocalPort 18000 -State Listen

# Kill the process
Stop-Process -Id <PID>

# Or use a different port by editing the profile
cc-manage edit <profile> # Change ProxyPort
```

“settings.json” conflicts cc-manage manages certain settings in ~/.claude/settings.json. If you see conflicts:

```
cc-manage settings repair
```

This removes ANTHROPIC\_API\_KEY, ANTHROPIC\_BASE\_URL, ANTHROPIC\_MODEL, and related overrides from settings.json while preserving other settings.

#### Provider returns errors

1. Check your API key is valid: `cc-manage key list`
2. Test the profile: `cc-manage test <profile> --level basic`
3. Check proxy logs (if debug enabled):

```
$env:GEMINI_PROXY_DEBUG = "1"    # For Gemini proxy
```

```
cc-manage models groq --refresh  
# If it fails, check your API key has model-list permissions
```

## Models not loading (dynamic providers)

### Running Tests

```
# Proxy conversion tests (no API keys needed)  
$env:CLAUDE_PROFILES_ROOT = "$PWD\src\cc-manage"  
python tests\test_proxy_conversions.py
```

---

## Uninstall

### Backup First

```
# Back up profiles and keys  
Copy-Item "$HOME\.claude-profiles" "$HOME\.claude-profiles.backup" -Recurse
```

### Remove on Windows

```
$installDir = Join-Path $HOME ".claude-profiles"  
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")  
$newUserPath = (($userPath -split ";") | Where-Object { $_ -and $_ -ne $installDir }) -join ";"  
[Environment]::SetEnvironmentVariable("Path", $newUserPath, "User")  
if (Test-Path -LiteralPath $installDir) {  
    Remove-Item -LiteralPath $installDir -Recurse -Force  
}
```

### Remove on macOS/Linux

```
rm -rf "$HOME/.claude-profiles"  
# Remove PATH entry from ~/.zshrc, ~/.bashrc, or ~/.profile:  
# # cc-manage PATH  
# export PATH="$HOME/.claude-profiles:$PATH"
```

### Clean Up Claude Settings

```
cc-manage settings repair    # Remove cc-manage overrides from settings.json
```